Naseeb Lal Market - Commercial Property Ecosystem

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

In commercial markets and complex retail spaces, managing monthly rent collections, electrical sub-meter utility readings, and municipal taxes across dozens of units is overwhelmingly manual and prone to severe revenue leakage. Landlords face persistent friction due to fraudulent online transaction claims, fake UPI screenshots, misplaced paper slips, and cumbersome dispute resolution.

The Solution: Naseeb Lal Market is an integrated commercial property management suite that pairs a native Android Admin Application (for property owners and collectors) with a zero-barrier, mobile-first Web Portal (for tenants). Through real-time cloud synchronization, automated utility calculations, and an anti-fraud reconciliation queue, it establishes an airtight, paperless property workflow.

2. SYSTEM ARCHITECTURE & TECHNOLOGY STACK

- Native Mobile Frontend: 100% Jetpack Compose with Material 3 dynamic styling, custom canvas graphics, and adaptive edge-to-edge layouts.
- Programming Language: Modern Kotlin 2.x utilizing Coroutines, StateFlow, and Clean MVVM architecture.
- Cloud Database: Google Firebase Realtime Database (RTDB) providing sub-second bidirectional synchronization, local disk persistence, and optimistic offline updates.
- Tenant Web Portal: Zero-dependency, high-speed HTML5 / CSS3 / ES6 web portal deployed via Firebase Hosting CDN with query-slug routing (?shop=X).
- Communication & SMS: Fast2SMS REST gateway integration for automated 1-click batch SMS reminders with native SIM selection fallback; WhatsApp direct deep-link dispatch.
- Document Engine: Android native PdfDocument vector engine and Apache POI spreadsheet generator for automated tax clearance slips and audit logs.

3. CORE FUNCTIONAL MODULES

A. Dual-Workspace Management

Features 1-tap workspace switching between Public Commercial Market (retail shops) and Private Residential Flats, providing segregated analytics while maintaining single database consistency.

B. Role-Based Access Control (RBAC)

Super-Admin maintains full administrative control over rent corrections, activity audits, and sub-admin accounts. Field Sub-Admins are limited strictly to recording payments and logging meter readings.

C. Utility Meter & Municipal Tax Billing Engine

Automates electricity sub-meter calculation using the formula: Due = (Current Reading - Previous Reading) x Tariff Rate. Tracks Patna Municipal Corporation (PMC) holding tax clearances with automated reminder windows.

D. Anti-Fraud Payment Verification Pipeline (Tenant Echo)

Eliminates fake payment claims. When a tenant enters an online UTR or cash note on the web portal, the form immediately locks to prevent duplicates. The claim is routed to the Admin dashboard's dedicated QUEUE badge. Only after manual bank reconciliation does the admin tap 'Verify & Mark Full Paid', which cryptographically provisions a tamper-evident receipt number and unlocks the official receipt on the tenant's browser.

4. INTERVIEW PRESENTATION PLAYBOOK (FOR CANDIDATES)

The 30-Second Elevator Pitch

How to introduce this project in an interview:

"I designed and developed Naseeb Lal Market, a full-stack commercial property management suite built with Jetpack Compose on Android and a real-time web portal on Firebase. It solves real-world rent tracking, utility sub-meter calculations, and payment fraud across commercial complexes. Its standout capability is an Anti-Fraud Verification Pipeline: tenants submit UTR proofs via zero-install deep links, and upon admin approval in the Android app, verified PDF receipts unlock dynamically in real time without refreshing the page."

The 2-Minute In-Depth Walkthrough (STAR Method)

- Situation (Problem): Commercial landlords struggle with manual notebooks, disputed utility calculations, and fraudulent UPI payment claims from retail tenants.
- Task (Objective): Build a high-reliability, fraud-proof ecosystem that requires zero onboarding friction for non-tech-savvy tenants while giving property owners real-time ledger visibility.
- Action (Engineering): Implemented clean MVVM on Android using Kotlin Coroutines and StateFlow for state management; established bidirectional WebSocket sync via Firebase RTDB; built a lightweight responsive tenant portal with automatic state polling; and implemented an on-device PDF generation engine.
- Result (Impact): Completely eliminated billing reconciliation disputes, enabled automated 1-click batch reminder dispatch, and achieved instant verified digital receipt generation.

5. TOP TECHNICAL INTERVIEW QUESTIONS & ANSWERS

Q1: Why did you choose Jetpack Compose over traditional XML Layouts?

Compose uses a declarative paradigm where UI is a pure function of state (UI = f(State)). In this application, real-time sync indicators, meter reading cards, and payment queue badges undergo frequent live state mutations. With Compose and StateFlow (observed via collectAsStateWithLifecycle), the framework performs intelligent recomposition on only the modified nodes. This eliminated complex View-binding boilerplate, reduced lines of code by over 40%, and completely eradicated null-pointer exceptions.

Q2: How did you achieve real-time synchronization between Android and the Web Portal?

We leveraged Firebase Realtime Database as the shared Single Source of Truth. The Android app maintains persistent WebSocket streams that emit snapshots into StateFlow pipelines. When a tenant submits a UTR on the web portal, an atomic REST PATCH updates the /rents/{id} node with tenantClaimedPaid: true. The Android dashboard immediately reflects this in the Queue filter. When approved, the status transitions to PAID. The web portal features an active polling observer that detects the change, transitions out of the pending state, and renders the official receipt.

Q3: Why a Web Portal for tenants instead of a Tenant Mobile App?

This was a deliberate UX and product strategy decision. Retail shopkeepers resist installing 50MB mobile applications and frequently forget credentials. By building a mobile-first Web Portal accessible via query parameters (?shop=5), we achieved zero barrier to entry. The tenant taps a monthly WhatsApp link, reviews their bill, enters their transaction reference, and saves their receipt. It maximizes compliance and adoption.

Q4: How did you prevent fraudulent payment submissions?

We enforced a two-stage verification barrier. A tenant submission is strictly categorized as an unverified claim (UNDER_REVIEW) and locks the input fields. The transaction appears in the Admin's dedicated 'Queue' badge. The property manager verifies the exact UTR against their live bank statement. Only when the manager clicks 'Verify & Mark Full Paid' does the system generate the official receipt number (NLM-YYYY-UNIT-XXXX) and unlock the sealed digital receipt on the tenant's device.

Q5: How does the system handle offline environments and network dropouts?

The Android app employs Firebase local disk persistence and SharedPreferences-backed data caching. If a field collector records a cash payment in a basement with zero connectivity, the local ledger updates immediately without freezing the UI. Once network connectivity is restored, Firebase's offline queue automatically synchronizes the transactions with cloud timestamps and activity audit logs.